

ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ №3

Філіпової Єлизавети 301 група

Тема: Побудова діаграми класів (Class Diagrams)

Предметна область: Інформаційна система курсів іноземних мов

1. Побудова діаграм класів (Class Diagrams)

Мета роботи: виконати побудову діаграми класів для проектованої інформаційної системи.

ЗАВДАННЯ

1. Для всіх об'єктів на діаграмах взаємодії призначити (створити) певний клас; для кожного повідомлення призначити (створити) відповідний метод (операцію) для класу об'єкта-приймача.
2. Розташувати створені класи з переліком операцій на діаграмі класів.
3. Для кожної операції визначити атрибути, які вона використовує та при необхідності додати їх до списку атрибутів класу.
4. Для кожного атрибуту задати логічний тип даних, для кожної операції логічний тип даних для return value та для переліку аргументів, якщо вони присутні.
5. Пов'язати класи на діаграмі класів, використовуючи різні типи відношень (асоціацію, агрегацію, композицію, наслідування).

Вимоги до виконання

1. Діаграма класів повинна містити не менше 10 класів.
2. Для кожного класу визначити не менше 5 атрибутів та 5 операцій.
3. По можливості використати всі типи відношень між класами.

Опис розробленої структури класів:

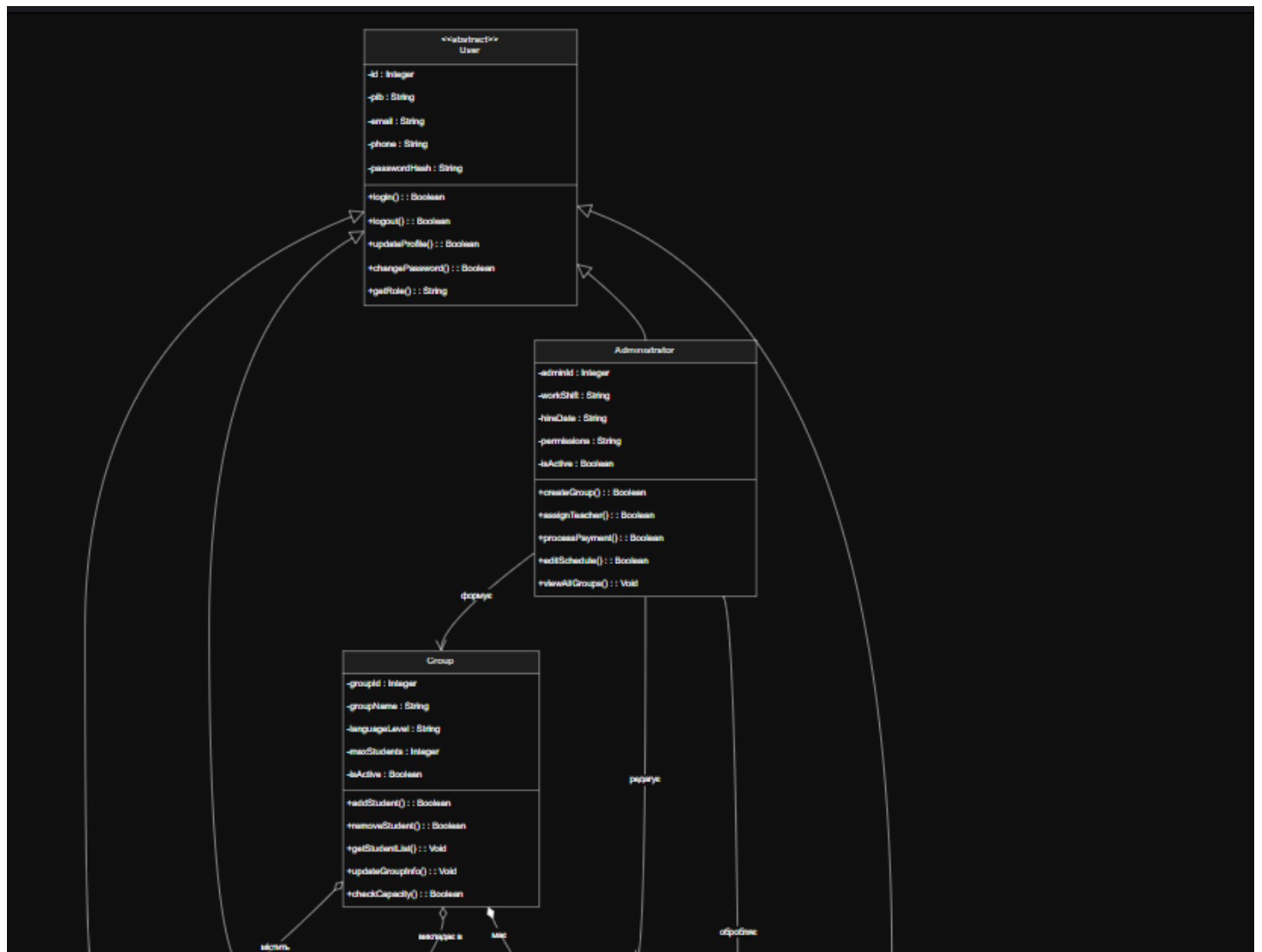
1. **User (Користувач):** Абстрактний базовий клас системи, що інкапсулює загальні властивості користувачів (id, pib, email, phone, passwordHash) та базові методи автентифікації.
2. **Student (Студент):** Клас-наслідник користувача. Зберігає інформацію про поточний рівень мови, баланс рахунку, номер студентського квитка та дату реєстрації. Містить методи проходження тестів та перегляду фінансів/розкладу.
3. **Teacher (Викладач):** Рольовий клас-наслідник. Містить дані про кваліфікацію, спеціалізацію, ставку заробітної плати та рейтинг. Забезпечує роботу з оцінюванням студентів та відміткою відвідуваності.

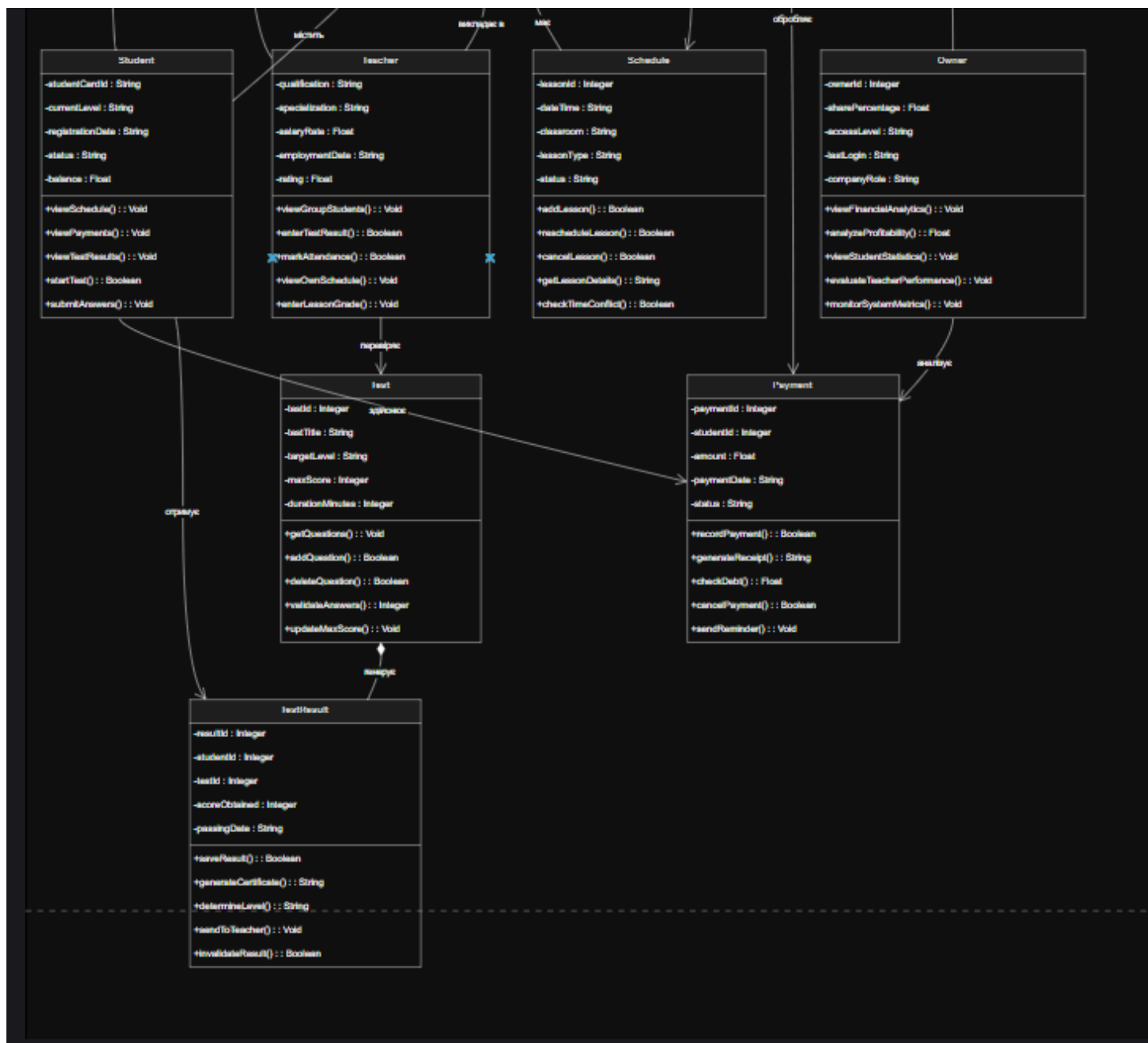
4. **Administrator (Адміністратор):** Клас керування системою. Містить атрибути shift, hireDate, permissions. Відповідає за створення груп, призначення викладачів та редагування розкладу.
5. **Owner (Власник курсів):** Спеціалізований клас для бізнес-аналітики. Оперує відсотком часток компанії та має методи для перегляду фінансової звітності, оцінки ефективності викладачів.
6. **Group (Навчальна група):** Сутність, що об'єднує студентів та викладачів за певним мовним рівнем. Керує лімітом місць у групі.
7. **Schedule (Розклад занять):** Клас, що описує проведення уроків, фіксує кабінети, час, типи занять (онлайн/офлайн) та статус заняття.
8. **Test (Тест):** Сутність для оцінювання знань. Зберігає назву тесту, тривалість, цільовий рівень мови та максимальний бал.
9. **TestResult (Результат тесту):** Клас для фіксації індивідуального результату студента за конкретний тест із методами генерації сертифікатів.
10. **Payment (Оплата):** Фінансова сутність, що реєструє суму, дату та поточний статус транзакції (Paid/Pending).

Опис відношень між класами та підписів зв'язків:

З метою деталізації логіки взаємодії об'єктів системи, усі зв'язки (окрім наслідування) було підписано текстовими маркерами із зазначенням напрямку читання дії:

- **Наслідування (Generalization):** класи Student, Teacher, Administrator, Owner успадковують базову структуру від абстрактного класу User (позначається лінією з порожнім білим трикутником біля батьківського класу).
- **Агрегація (Aggregation):** клас Group агрегує класи Student (підпис: містить) та Teacher (підпис: викладає в), оскільки видалення групи не знищує сутності користувачів у загальній базі даних (біля класу Group розміщено порожній ромб).
- **Композиція (Composition):** жорстка залежність життєвого циклу реалізована між Group та Schedule (підпис: має), а також між Test та TestResult (підпис: генерує). Видалення головного класу автоматично знищує залежні (біля головного класу розміщено зафарбований ромб).
- **Спрямована асоціація (Directed Association):** звичайні бізнес-дії між сутностями системи, позначені тонкою стрілкою:
 - Student $\rightarrow$ Payment (підпис: здійснює)
 - Student $\rightarrow$ TestResult (підпис: отримує)
 - Teacher $\rightarrow$ Test (підпис: перевіряє)
 - Administrator $\rightarrow$ Group (підпис: формує)
 - Administrator $\rightarrow$ Schedule (підпис: редагує)
 - Administrator $\rightarrow$ Payment (підпис: обробляє)
 - Owner $\rightarrow$ Payment (підпис: аналізує)





2. Побудова діаграми станів та переходів (Statechart Diagrams) Мета роботи: виконати побудову діаграми станів та переходів (statechart diagrams).

ЗАВДАННЯ Створити одну діаграму станів для опису процесу функціонування обраної системи в цілому і дві діаграми для конкретних елементів системи. Використовувати діаграму станів для авторизації користувачів забороняється.

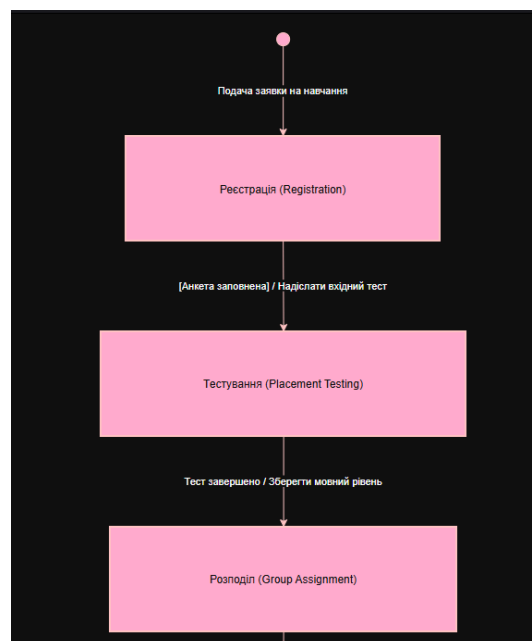
Вимоги

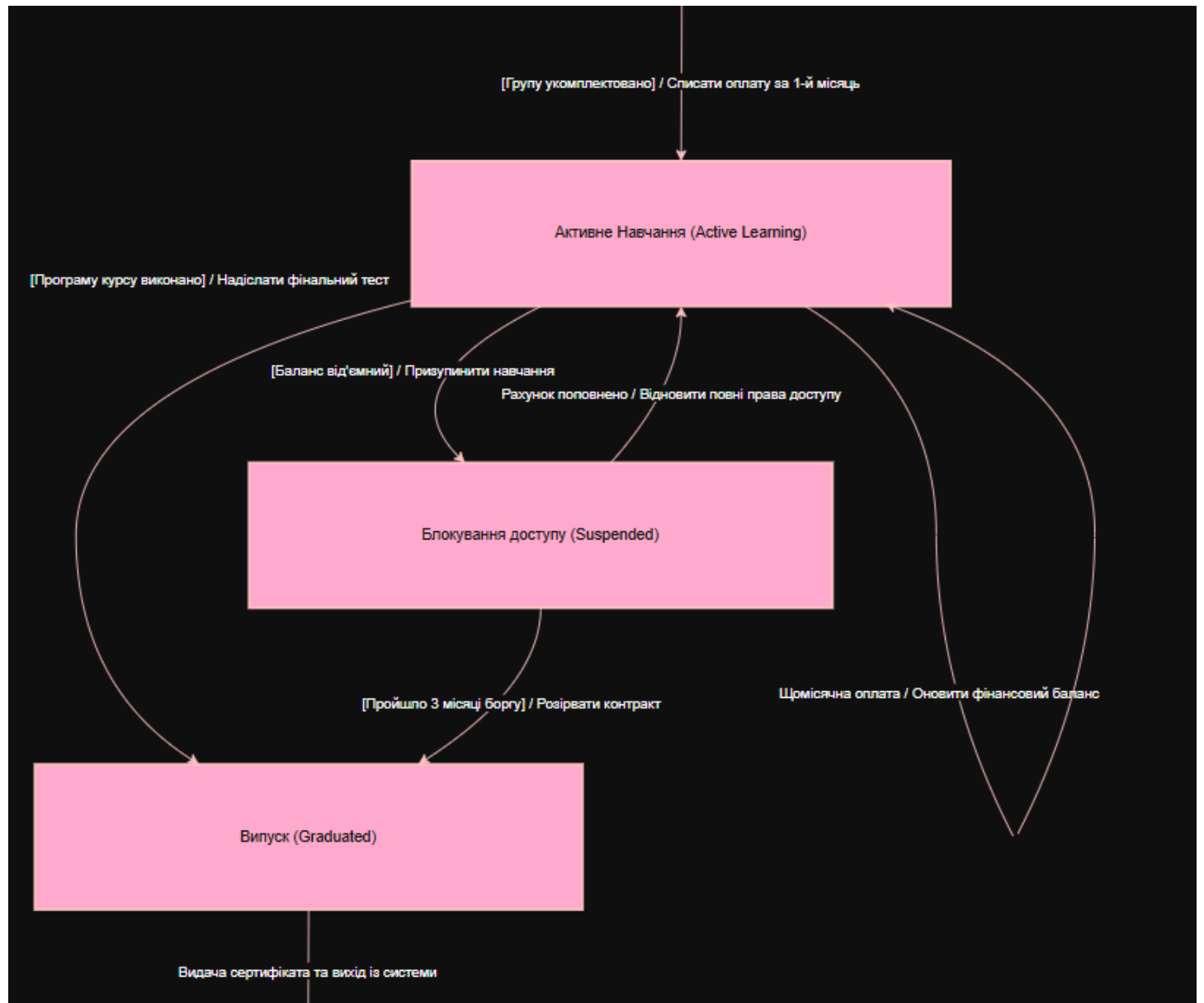
1. Кожна діаграма повинна містити не менше 6 станів.
2. По можливості використати обидва типи переходів (звичайний і рефлексивний).
3. Для кожного переходу визначити хоча б одну з характеристик (тригер, гранична умова, дія). Теоретичні відомості

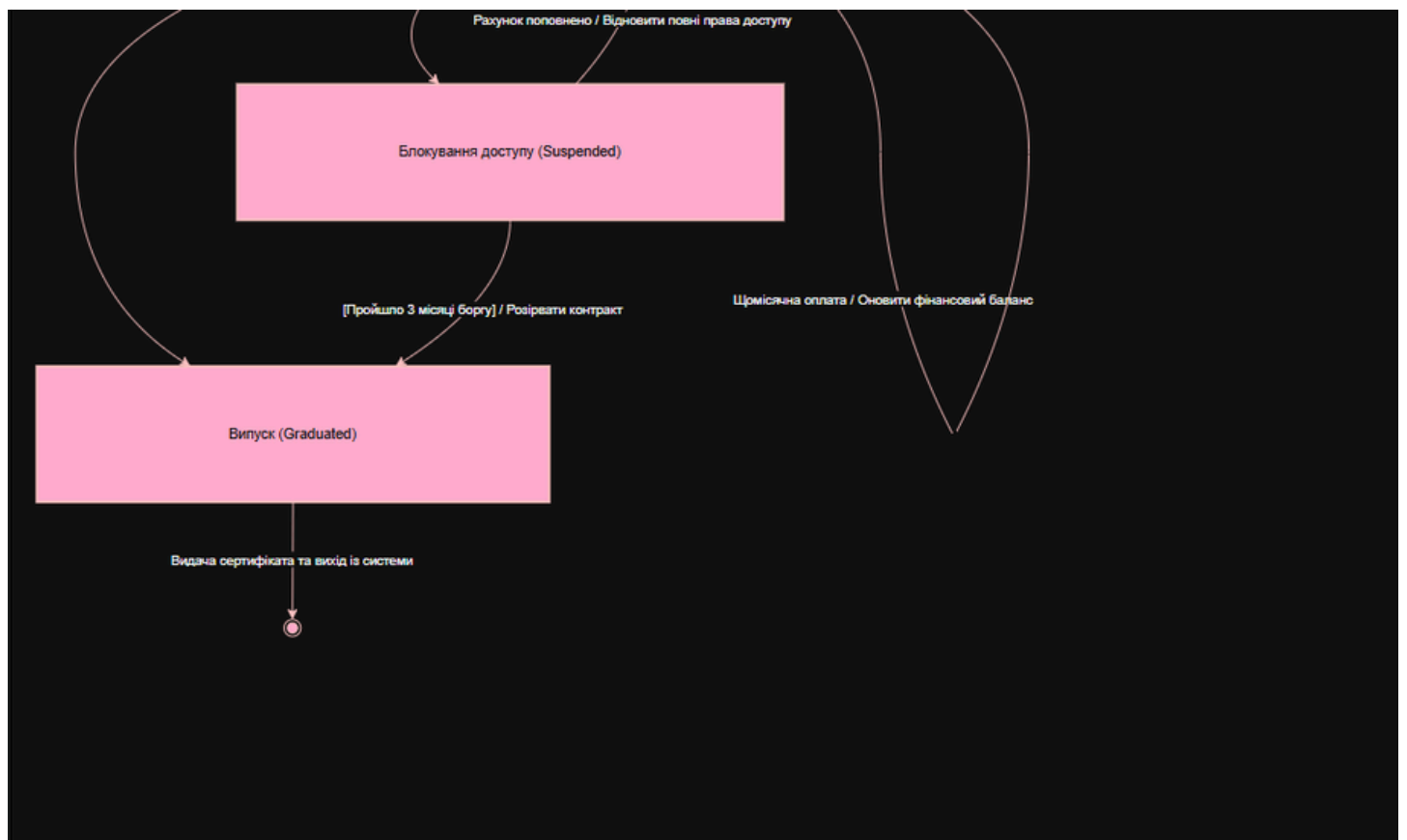
1)Опис об'єкта проектування: Об'єктом моделювання обрано ключову сутність системи — **Студент (Student)**. Діаграма відображає динаміку зміни його статусів в інформаційній системі мовної школи, починаючи від первинного звернення (подачі заявки) і закінчуючи успішним випуском або розірванням договору.

Аналіз станів системи (6 базових станів):

- **Реєстрація (Registration):** Початковий етап, на якому при вході (entry/) створюється картка, під час роботи (do/) очікуються дані, а при виході (exit/) присвоюється ID.
- **Тестування (Placement Testing):** Стан визначення рівня знань. При вході генерується доступ до тесту, а при виході фіксується рівень (A1–C2).
- **Розподіл (Group Assignment):** Етап очікування формування групи адміністратором з урахуванням графіка студента.
- **Активне Навчання (Active Learning):** Основний довготривалий стан, де студент відвідує заняття та отримує оцінки.
- **Блокування доступу (Suspended):** Тимчасовий стан обмеження прав через заборгованість, у якому система виконує регулярне сповіщення про борг.
- **Випуск (Graduated):** Кінцевий успішний стан, що супроводжується автоматичним генераційним процесом створення сертифіката.



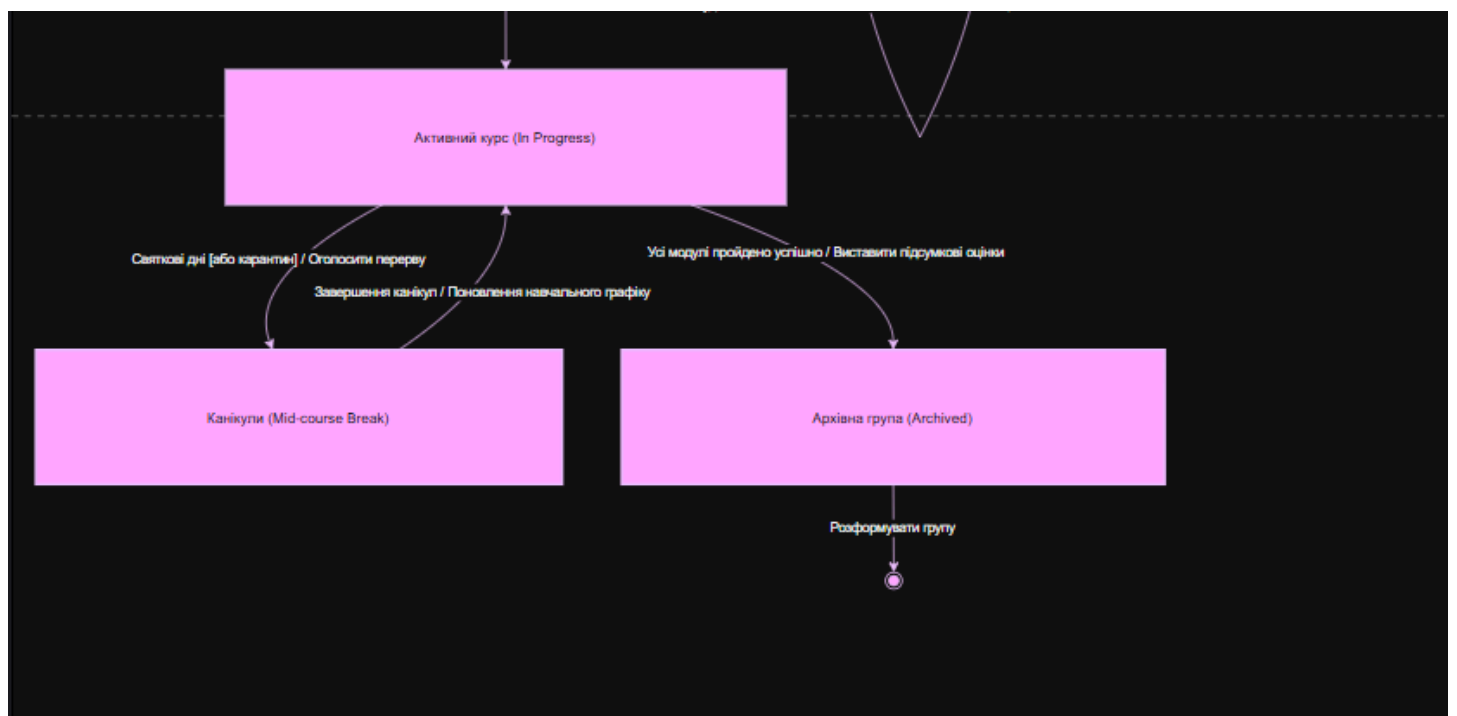
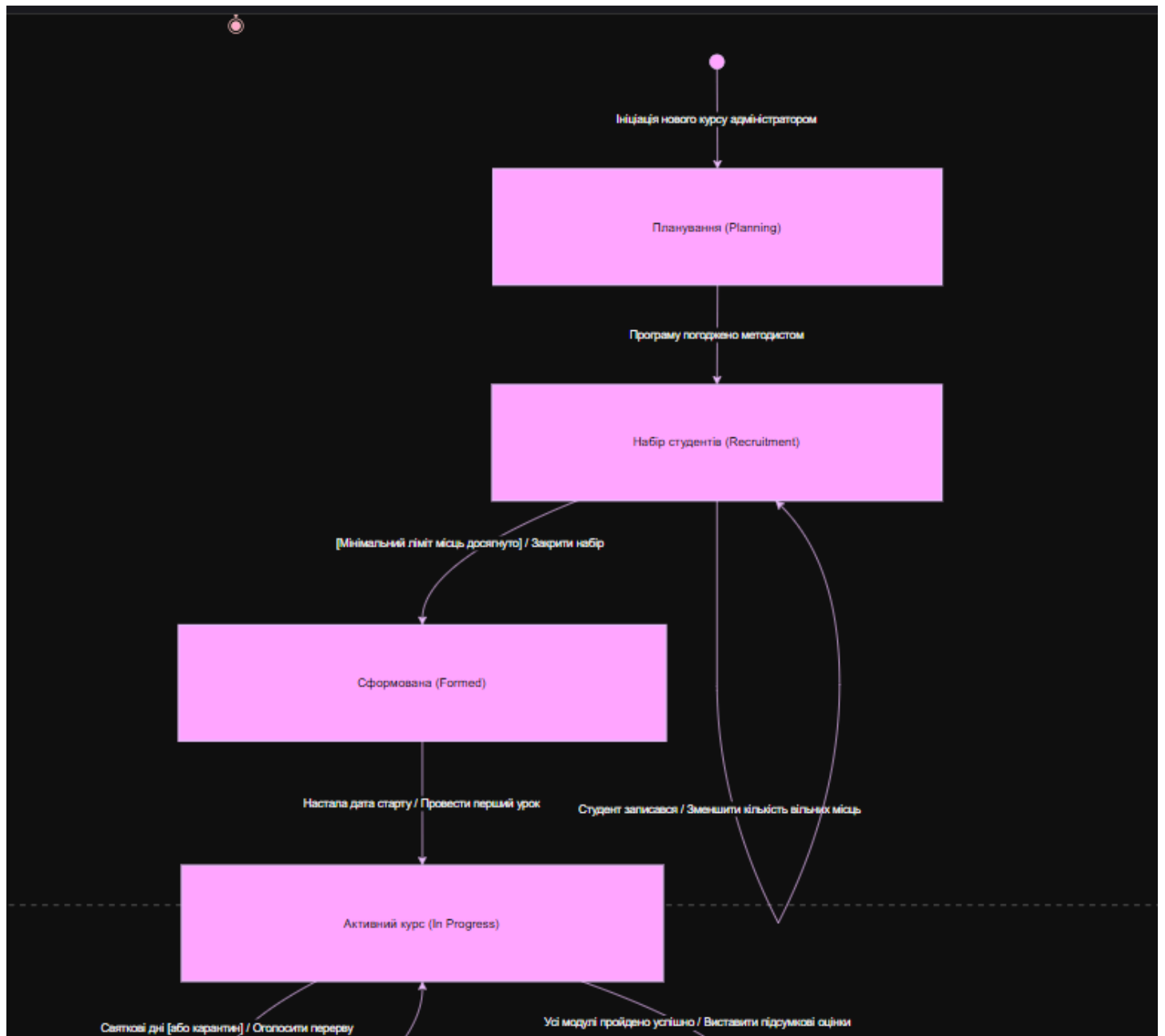




2)Опис об'єкта проектування: Об'єктом моделювання є організаційна сутність — **Навчальна група (Group)**. Діаграма описує процеси адміністративного менеджменту груп: від планування розкладу методистом до архівування після завершення навчального семестру.

Аналіз станів системи (6 базових станів):

- **Планування (Planning):** Створення картки групи, затвердження мовної програми та базових годин занять.
- **Набір студентів (Recruitment):** Відкритий статус, коли група стає доступною для запису на сайті. Система моніторить кількість заявок (do/).
- **Сформована (Formed):** Стан готовності. При вході (entry/) автоматично закріплюється викладач та аудиторія, студенти отримують пуш-сповіщення про дату старту.
- **Активний курс (In Progress):** Стан проведення уроків, ведення електронного журналу та фіксації відвідуваності.
- **Канікули (Mid-course Break):** Тимчасова зупинка занять (наприклад, святкові дні), що зберігає поточне навантаження викладача.
- **Архівна група (Archived):** Кінцевий етап, коли картка закривається для редагування, а статистика успішності передається в архів.

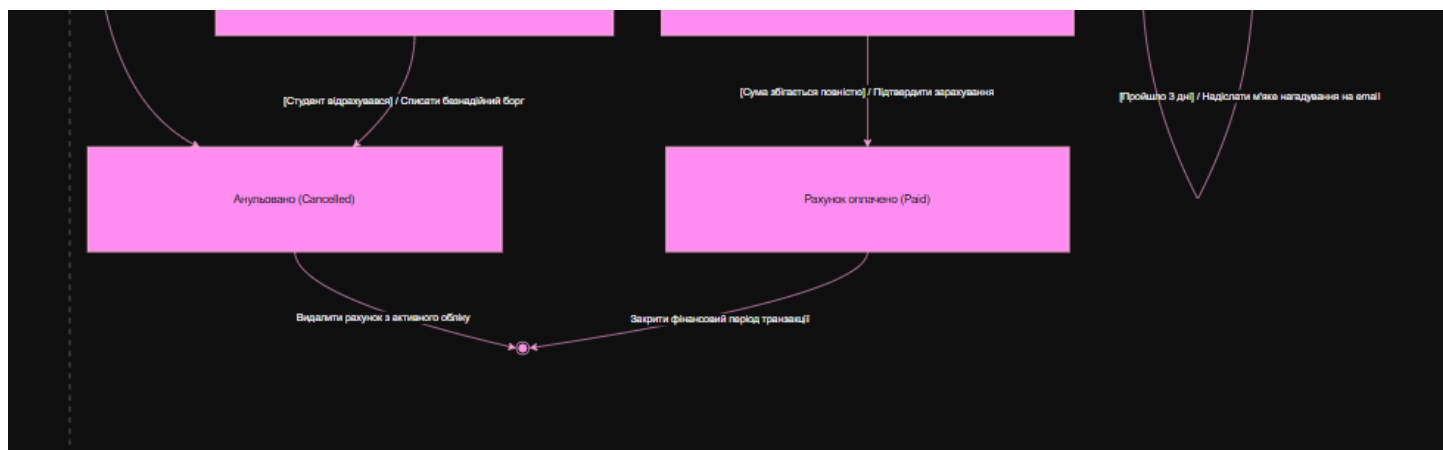


3)Опис об'єкта проектування: Об'єктом моделювання виступає фінансова сутність системи — **Рахунок на оплату (Payment)**. Діаграма відображає транзакційні стани білінгової підсистеми мовної школи, яка взаємодіє із зовнішнім банківським еквайрингом та балансом користувача.

Аналіз станів системи (6 базових станів):

- **Генерація рахунку (Draft):** Початкова стадія розрахунку вартості за обраним студентом тарифом та присвоєння UUID.
- **Очікування оплати (Pending):** Рахунок виставлено. Під час перебування в цьому стані система очікує на відповідь від банку.
- **Верифікація (Verifying):** Проміжний стан безпеки, у якому білінг зчитує платіжні логи та перевіряє цілісність токена.
- **Рахунок оплачено (Paid):** Успішний фінал. Кошти зараховуються на баланс студента, генерується фіскальний чек.
- **Протерміновано (Overdue):** Фіксація боргу у разі порушення дедлайну, що ініціює надсилання критичних попереджень.
- **Анульовано (Cancelled):** Стан скасування платіжної вимоги через помилку або відрахування студента.





3. Побудова діаграми діяльності (Activity Diagrams) Мета роботи: виконати побудову діаграми діяльності (activity diagrams).

ЗАВДАННЯ

Побудувати 3 діаграми діяльності для окремих варіантів використання системи.

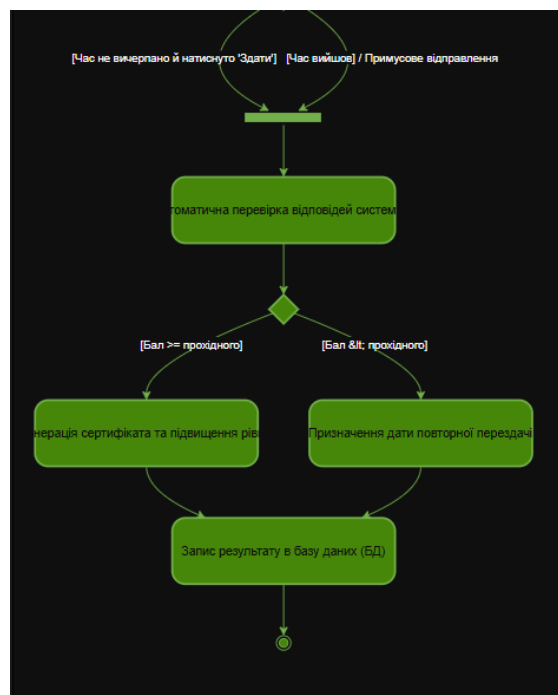
Вимоги

1. Кожна діаграма повинна містити не менше 6 діяльностей.
2. При побудові кожної діаграми використовувати стани прийняття рішення та синхронізації.

1) **Об'єкт моделювання:** Варіант використання «Пройходження тестування знань Студентом».

Опис діяльностей: Студент проходить авторизацію та обирає необхідний тест. Після цього за допомогою **лінійки синхронізації (Fork)** процес розгалужується на дві паралельні потокові діяльності: паралельно із запуском таймером обмеження часу студент виконує введення відповідей на запитання.

Логіка розгалужень: Вузол рішення `is_time_over` перевіряє часові ліміти. Обидва паралельні процеси зливаються через **лінійку синхронізації (Join)**, після чого система оцінює роботу. Наступний вузол вибору `is_passed` аналізує успішність: у разі набрання прохідного балу діяльність переходить до генерації сертифіката, інакше — призначається дата перездачі. Кінцевою дією є збереження логів у базу даних.



2)Об'єкт моделювання: Варіант використання «Оплата за навчання та активація послуг».

Опис діяльності: Процес починається з виставлення інвойсу. Вузол прийняття рішень `check_method` розділяє логіку на два альтернативні шляхи: електронний платіж через інтернет-еквайринг або фізичне внесення коштів через адміністратора школи.

Логіка розгалужень: Після успішного підтвердження платежу (автоматичного чи ручного) процес підходить до **лінійки синхронізації (Fork)**. Система паралельно виконує

дві незалежні бізнес-операції: миттєво оновлює фінансовий баланс у картці студента та одночасно надсилає запит на генерацію офіційного фіскального чека. Обидва потоки зливаються у **лінійці синхронізації (Join)**, після чого клієнту відправляється квитанція та відкривається повний допуск до занять.



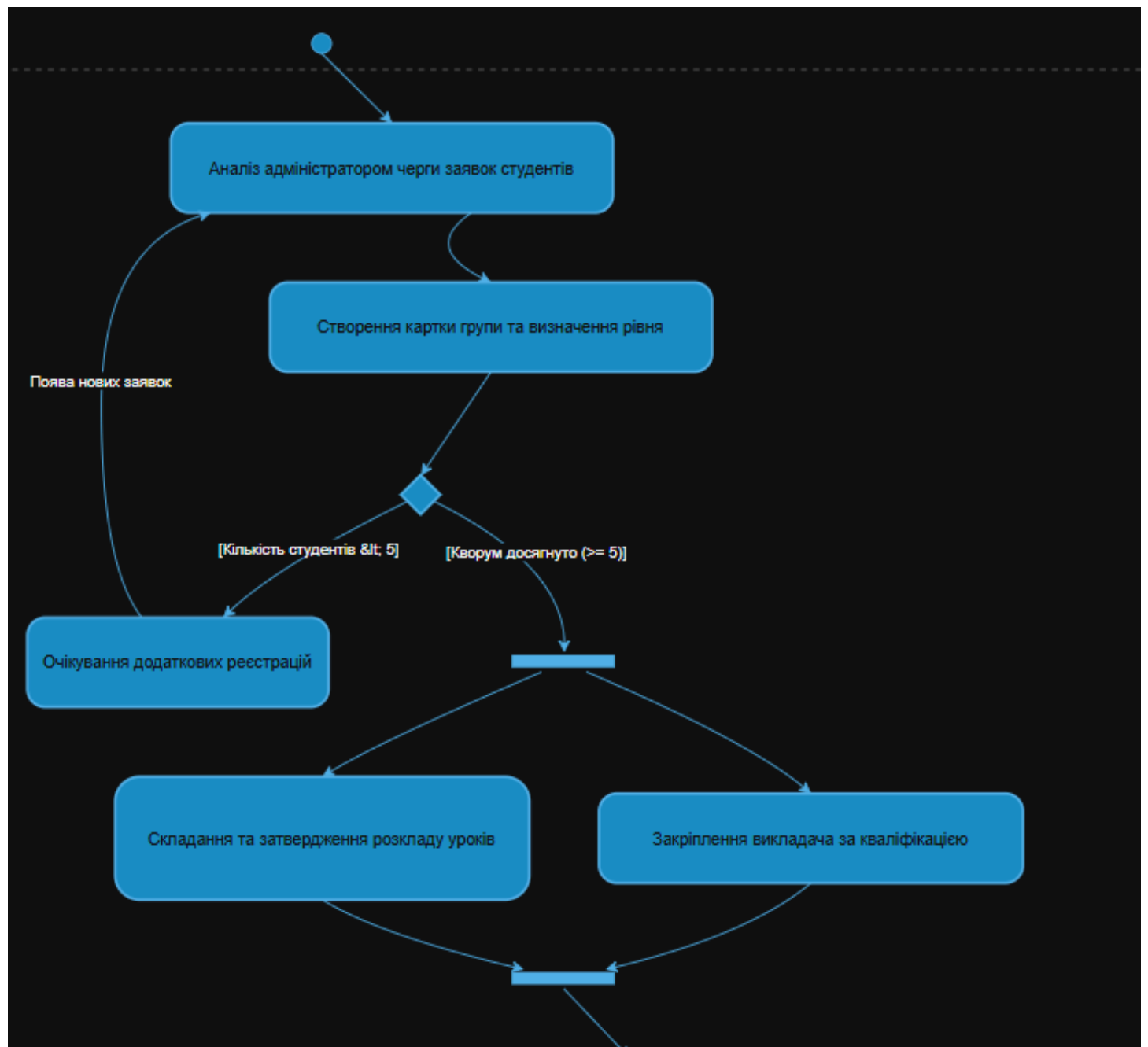


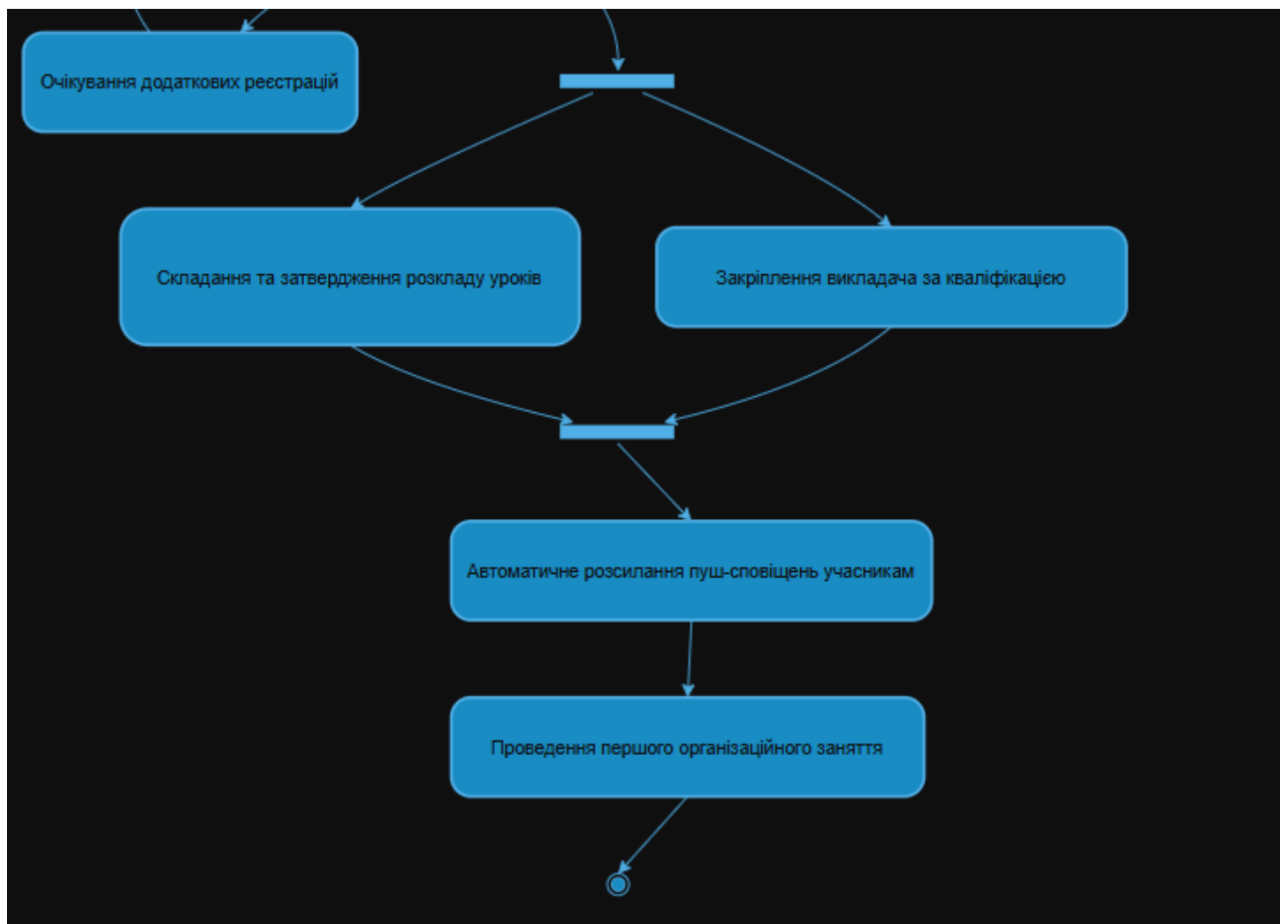
3) **Об'єкт моделювання:** Варіант використання «Формування та запуск мовної групи».

Опис діяльностей: Адміністрація аналізує накопичені заявки та ініціює створення картки групи під конкретний рівень мови (наприклад, English B2). Вузол вибору `check_quorum` перевіряє наявність мінімальної кількості людей (5 осіб) для старту економічно вигідного курсу. Якщо людей недостатньо, група переходить у режим очікування додаткових реєстрацій.

Логіка розгалужень: Коли кворум набрано, спрацьовує **лінійка синхронізації (Fork)**. Адміністратор одночасно виконує дві організаційні задачі: підбирає вільного викладача відповідної кваліфікації та паралельно резервує навчальні години й кабінети у модулі розкладу. Процеси об'єднуються **лінійкою синхронізації (Join)**, що забезпечує запуск

розсилки сповіщень студентам лише після того, як і викладача, і розклад успішно призначено. Завершується діяльність першим уроком.





4. Побудова діаграм компонентів (Component Diagrams) Мета роботи: виконати побудову діаграм компонентів (component diagrams).

ЗАВДАННЯ

Побудувати діаграму компонентів для обраної програмної системи. Діаграма повинна містити не менше трьох компонентів. Розподілити всі класи між компонентами.

Опис компонентів: Система побудована за три рівневою клієнт-серверною архітектурою:

1. **Клієнтський рівень:** * LanguageSchool.exe — графічний інтерфейс користувача (GUI) для студентів та адміністрації.
2. **Серверний рівень (Бізнес-логіка):**
 - AuthService.dll — модуль авторизації та безпеки.
 - EduManager.dll — модуль керування навчанням (групи, розклад, тести).
 - BillingService.dll — фінансовий модуль (рахунки, оплата).
3. **Рівень даних та зовнішні сервіси:**
 - CourseSystem_DB — основна база даних системи.
 - BankAcquiring_API — зовнішній банківський шлюз для онлайн-оплат.

Взаємодія: Клієнтський застосунок (.exe) надсилає запити до серверних бібліотек (.dll). Серверні модулі обробляють логіку, виконують запити до бази даних (CourseSystem_DB), а фінансовий модуль додатково звертається до зовнішнього банку (BankAcquiring_API) для проведення транзакцій.

